

2.3. Les messages dans leur propre fichier

Les messages et leur place vont dans « `variables.h` » ; `main()` ne garde que les boutons.

1. Ce qu'il faut comprendre

Comme en 1.11 : chaque message devient un `Mot`, rangé dans `variables.h`. C'est l'onglet à côté de `principal.cpp`.

`principal.cpp` ne parle plus que des **boutons** : A efface `bonjour`, B le remet. Les places et les textes ne s'y écrivent plus du tout.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1372 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

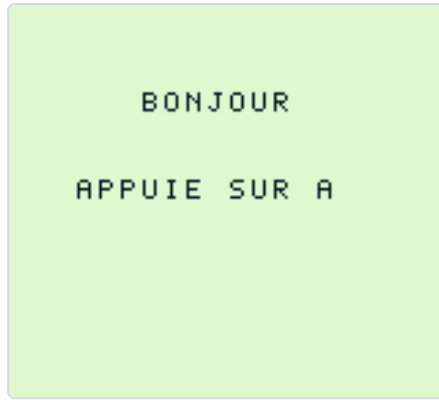
```
int main() {  
    texte(bonjour);  
    texte(consigne);  
  
    while (true) {  
        image();  
  
        if (bouton(A)) effacer(bonjour);  
        if (bouton(B)) texte(bonjour);  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



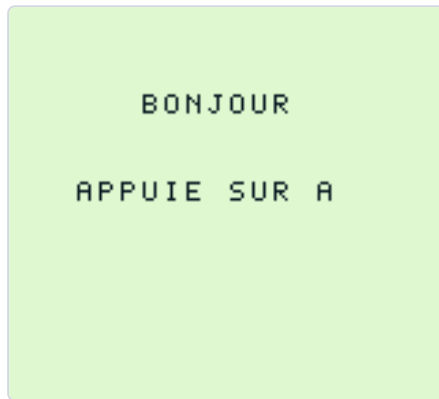
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé



B — l'écran a changé

5. Ce qu'on doit voir

A efface « BONJOUR » ; B le remet. La consigne reste.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ « BONJOUR » est là au départ
- ✓ A l'efface entièrement
- ✓ B le remet
- ✓ la consigne ne bouge pas

Ensuite : **3. Un carré de 8 × 8** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++